

CivicConnect Platform

TABLE OF CONTENTS

Comprehensive Project File Structure & Core Source Code Reference

1. System Architecture & High-Level Technology Stack	Page 2
2. Complete Project File Directory Hierarchy	Page 2
3. Frontend Application Routing & Role Gates (src/App.jsx)	Page 3
4. Production Server Entry & Express API (server/index.js & functions/index.js)	Page 4
5. Database Layer & PostgreSQL Schema Client (functions/db.js)	Page 5
6. Workflow State Machine & Finite Role Definitions (src/constants/workflow.js)	Page 6
7. AI/ML Categorization & Duplicate Detection Pipeline (functions/ai.js)	Page 7
8. REST API Contracts, Endpoints & Security Specification	Page 8

Document Overview

- System: Citizen Grievance Redressal & Smart City AI Dispatcher
- Tech Stack: React 19, Vite, Tailwind CSS, Node.js/Express, Supabase (PostgreSQL), Firebase Auth
- AI/ML Engine: Google Gemini 2.0 / GenAI, TF-IDF Text Classification, Haversine Duplicate Detection
- Architecture: Clean 3-Tier Multi-Tenant SaaS with Realtime State Machine & GIS Geofencing
- Generated Date: October 2026 | Production Build Release v1.0.0

SECTION 1

System Architecture & High-Level Tech Stack

CivicConnect is built as an enterprise-grade civic grievance redressal portal. The system uses a modern 3-tier architecture with Supabase PostgreSQL as the primary single-source-of-truth datastore, Firebase Authentication for role-based JWT issuance, Google Gemini AI for automated categorization and severity scoring, and React 19 for a high-performance, accessible citizen/admin UI.

Core Technology Components

- Frontend: React 19, Vite 6, Tailwind CSS 4, Framer Motion, Leaflet GIS Maps, Recharts, Lucide Icons
- Backend API: Node.js / Express 5 with modular micro-handlers (Cloud Functions / Containerized)
- Database & Storage: Supabase PostgreSQL (Structured relational schema) + Cloudinary Media Storage
- AI Engine: Google Gemini 2.0 / GenAI SDK, TF-IDF NLP model, Haversine Spatio-Temporal Duplicate Filter
- Realtime & Auth: Firebase Auth (RBAC Custom Claims) + Supabase Realtime WebSocket Channels

SECTION 2

Complete Project Directory Structure

```
civicnew-main/
% % % .agents/           # AI Assistant skills & custom guidelines
% % % ai/                # Machine Learning Training, Datasets & Models
% % % dataset/          # Categorization and severity labeled data
% % % model/            # Serialized classifier weights
% % % src/              # Training & inference Python scripts
% % % functions/        # Modular Express API & Cloud Functions
% % % index.js          # API Router & Express application configuration
% % % authMiddleware.js  # Firebase JWT verification & role authorization
% % % db.js             # Supabase PostgreSQL client & query engine
% % % ai.js             # Gemini AI triage & automated classification
% % % complaintsHandler.js # Complaint lifecycle CRUD & status transition
% % % departmentRoutingService.js # Department dispatch & SLA escalation
% % % departmentsHandler.js # Department queues, engineers & analytics
% % % engineerHandler.js # Field engineer updates & photo verification
% % % adminHandler.js   # System analytics, user administration & logs
% % % duplicates.js     # Geolocation (Haversine) & similarity analysis
% % % workflow.js       # Complaint Finite State Machine validator
% % % server/           # Standalone Node.js production web server
% % % index.js          # Express HTTP bootstrap listener (Port 5177)
% % % src/              # React 19 Frontend Application (Vite SPA)
% % % main.jsx          # React DOM root mounting
% % % App.jsx           # Global route dispatcher & role protection
% % % firebase.js       # Firebase SDK client initialization
% % % components/       # Modular UI components (auth, admin, dept, ui)
% % % constants/        # Workflow status, SLA timers, and RBAC roles
% % % pages/            # Route views (Landing, Report, Dashboard, Admin)
% % % services/         # API integrations (Supabase, Cloudinary, AI)
% % % styles/           # Global Tailwind CSS and styling themes
% % % supabase/         # PostgreSQL schema migrations & DDL
% % % firestore.rules    # Security and authorization rules
% % % package.json      # Root dependency configuration & scripts
% % % vite.config.js     # Vite build pipeline & alias configurations
```

SECTION 3

Frontend Application Routing & Role Gates (src/App.jsx)

The frontend router controls application navigation and strictly enforces Role-Based Access Control (RBAC). Routes are partitioned into Public (Landing, Track, Auth), Citizen Protected (Grievance Submission, Tracking), Department Officer, Field Engineer, and Super Administrator views.

Source Code: src/App.jsx

```
import { Routes, Route, useLocation } from "react-router-dom";
import { AnimatePresence } from "framer-motion";
import AuroraBackground from "../components/layout/AuroraBackground.jsx";

// Citizen / Public Pages
import LandingPage from "../pages/LandingPage.jsx";
import LoginPage from "../pages/LoginPage.jsx";
import RegisterPage from "../pages/RegisterPage.jsx";
import DashboardPage from "../pages/DashboardPage.jsx";
import ReportIssuePage from "../pages/ReportIssuePage.jsx";
import TrackComplaintPage from "../pages/TrackComplaintPage.jsx";

// Role-protected Route Guards
import ProtectedRoute from "../components/auth/ProtectedRoute.jsx";
import RoleRoute from "../components/auth/RoleRoute.jsx";
import AdminRoute from "../components/auth/AdminRoute.jsx";
import { ROLES } from "../constants/workflow.js";

// Role-specific Dashboards
import AdminDashboardPage from "../pages/admin/Dashboard.jsx";
import DepartmentDashboard from "../pages/department/Dashboard.jsx";
import EngineerDashboard from "../pages/engineer/Dashboard.jsx";

export default function App() {
  const location = useLocation();

  return (
    <>
      <AuroraBackground />
      <AnimatePresence mode="wait">
        <Routes location={location} key={location.pathname}>
          {/* Public Routes */}
          <Route path="/" element={<LandingPage />} />
          <Route path="/login" element={<LoginPage />} />
          <Route path="/register" element={<RegisterPage />} />
          <Route path="/track" element={<TrackComplaintPage />} />

          {/* Citizen Protected Routes */}
          <Route path="/dashboard" element={<ProtectedRoute><DashboardPage /></ProtectedRoute>} />
          <Route path="/report" element={<ProtectedRoute><ReportIssuePage /></ProtectedRoute>} />

          {/* Department Protected Routes */}
          <Route path="/department/dashboard" element={<RoleRoute allowedRoles={[ROLES.DEPARTMENT]}
><DepartmentDashboard /></RoleRoute>} />
          {/* Engineer Protected Routes */}
          <Route path="/engineer/dashboard" element={<RoleRoute allowedRoles={[ROLES.ENGINEER]}><EngineerDashboard /
></RoleRoute>} />
          {/* Admin Protected Routes */}
          <Route path="/admin/dashboard" element={<AdminRoute><AdminDashboardPage /></AdminRoute>} />
        </Routes>
      </AnimatePresence>
    </>
  );
}
```

SECTION 4

Production Server & API Router (server/index.js & functions/index.js)

The backend utilizes Express 5 mounted over HTTP and Cloud Functions. It coordinates database persistence, Google Gemini AI triage, geospatial clustering, and Firebase user session validation.

Source Code: server/index.js

```
import express from "express";
import http from "http";
import { createRequire } from "module";

const require = createRequire(import.meta.url);
const { app } = require("../functions/index.js");

const PORT = process.env.PORT || 5177;
const server = http.createServer(app);

server.listen(PORT, "0.0.0.0", () => {
  console.log(" CivicConnect Production Backend API Server Online");
  console.log(` Port: ${PORT} (0.0.0.0)`);
  console.log(" Single Source of Truth: Supabase PostgreSQL & Firebase Auth");
});

export default app;
```

Source Code: functions/index.js (Route Registration Summary)

```
const express = require("express");
const cors = require("cors");
const { authenticateUser, requireRole } = require("../authMiddleware");
const complaintsHandler = require("../complaintsHandler");
const adminHandler = require("../adminHandler");
const engineerHandler = require("../engineerHandler");

const app = express();
app.use(cors({ origin: true, credentials: true }));
app.use(express.json());

// Public & Citizen Endpoints
app.post("/api/complaints/submit", authenticateUser, complaintsHandler.submitComplaint);
app.get("/api/complaints/my", authenticateUser, complaintsHandler.getUserComplaints);
app.get("/api/complaints/track/:trackingId", complaintsHandler.trackComplaint);

// Department & Engineer Workflow Endpoints
app.get("/api/department/complaints", authenticateUser, requireRole(["department", "admin"]),
  complaintsHandler.getDepartmentComplaints);
app.post("/api/engineer/postWorkUpdate", authenticateUser, requireRole(["engineer"]), engineerHandler.postWorkUpdate);

// Admin Control Endpoints
app.get("/api/admin/metrics", authenticateUser, requireRole(["admin"]), adminHandler.getSystemMetrics);
app.post("/api/admin/reassign", authenticateUser, requireRole(["admin"]), adminHandler.reassignComplaint);

module.exports = { app };
```

SECTION 5

Database Client & PostgreSQL Schema (functions/db.js)

All persistent business entities (Complaints, Users, Departments, Timeline Events, Audit Logs) are stored in Supabase PostgreSQL with strict relational foreign keys and indices.

Source Code: functions/db.js (Supabase Client & Query Layer)

```
const { createClient } = require("@supabase/supabase-js");

const SUPABASE_URL = process.env.SUPABASE_URL;
const SUPABASE_SERVICE_KEY = process.env.SUPABASE_SERVICE_ROLE_KEY;

if (!SUPABASE_URL || !SUPABASE_SERVICE_KEY) {
  console.error("CRITICAL: Missing Supabase environment credentials");
}

const supabase = createClient(SUPABASE_URL, SUPABASE_SERVICE_KEY, {
  auth: { persistSession: false, autoRefreshToken: false }
});

// Database Access Object (DAO)
const db = {
  // Complaints Collection
  async createComplaint(complaintData) {
    const { data, error } = await supabase.from("complaints").insert([complaintData]).select().single();
    if (error) throw error;
    return data;
  },

  async getComplaintById(id) {
    const { data, error } = await supabase.from("complaints").select("*, department:departments(*)").eq("id", id).single();
    if (error) throw error;
    return data;
  },

  async updateComplaintStatus(id, status, metadata = {}) {
    const { data, error } = await supabase.from("complaints").update({ status, updated_at: new Date().toISOString(), metadata }).eq("id", id).select().single();
    if (error) throw error;
    return data;
  }
};

module.exports = { supabase, db };
```

SECTION 6

Workflow State Machine & Constants (src/constants/workflow.js)

To guarantee data integrity and SLA adherence, all grievance transitions obey a deterministic Finite State Machine (FSM). Unauthorized skips (e.g., jumping from SUBMITTED directly to RESOLVED without assignment) are blocked at the schema and middleware level.

Source Code: src/constants/workflow.js

```
export const ROLES = {
  CITIZEN: "citizen",
  DEPARTMENT: "department",
  ENGINEER: "engineer",
  ADMIN: "admin"
};

export const COMPLAINT_STATUS = {
  SUBMITTED: "SUBMITTED",
  TRIAGED: "TRIAGED",
  ASSIGNED: "ASSIGNED",
  IN_PROGRESS: "IN_PROGRESS",
  RESOLVED: "RESOLVED",
  CLOSED: "CLOSED",
  REJECTED: "REJECTED",
  DUPLICATE: "DUPLICATE"
};

export const VALID_STATUS_TRANSITIONS = {
  SUBMITTED: ["TRIAGED", "REJECTED", "DUPLICATE"],
  TRIAGED: ["ASSIGNED", "REJECTED"],
  ASSIGNED: ["IN_PROGRESS", "TRIAGED"],
  IN_PROGRESS: ["RESOLVED", "ASSIGNED"],
  RESOLVED: ["CLOSED", "IN_PROGRESS"],
  CLOSED: [],
  REJECTED: [],
  DUPLICATE: []
};

export const SEVERITY_LEVELS = {
  LOW: { label: "Low", slaHours: 72, color: "#10b981" },
  MEDIUM: { label: "Medium", slaHours: 48, color: "#f59e0b" },
  HIGH: { label: "High", slaHours: 24, color: "#f97316" },
  CRITICAL: { label: "Critical", slaHours: 6, color: "#ef4444" }
};
```

SECTION 7

AI Categorization & Duplicate Detection (functions/ai.js)

CivicConnect utilizes Google Gemini 2.0 to automatically analyze incoming grievance descriptions, classify them into appropriate civic municipal departments, assign urgency levels, and evaluate geospatial proximity against existing complaints.

Source Code: functions/ai.js (Gemini AI Grievance Triage)

```
const { GoogleGenAI } = require("@google/genai");
const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });

async function categorizeComplaint(title, description, imageUrl = null) {
  const prompt = `
  Analyze this civic issue and return a structured JSON response:
  Title: "${title}"
  Description: "${description}"

  Format your response strictly as JSON with the following keys:
  - department: "Roads & Transport" | "Water Supply" | "Sanitation" | "Electricity" | "Public Works"
  - category: Short category name (e.g., "Pothole", "Pipeline Leak")
  - priority: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL"
  - confidence: number between 0.0 and 1.0
  - summary: one-sentence summary
  `;

  try {
    const response = await ai.models.generateContent({
      model: "gemini-2.0-flash",
      contents: prompt,
      config: { responseMimeType: "application/json" }
    });
    return JSON.parse(response.text);
  } catch (error) {
    console.error("AI Categorization Fallback:", error.message);
    return {
      department: "Public Works",
      category: "General Grievance",
      priority: "MEDIUM",
      confidence: 0.5,
      summary: title
    };
  }
}

module.exports = { categorizeComplaint };
```

SECTION 8

REST API Contracts & Production Security

Every request is subject to cryptographic validation, SQL injection prevention, rate limiting, and RBAC token evaluation.

Key REST API Endpoints Summary

Endpoint & Method	Authorization	Description
POST /api/complaints/submit	Bearer JWT (Citizen)	Submits new complaint; runs AI triage & duplicate check.
GET /api/complaints/track/:id	Public / Anonymous	Fetches complaint public timeline and real-time status.
PATCH /api/complaints/:id/status	Bearer JWT (Dept/Admin)	Applies state transition obeying FSM constraints.
POST /api/engineer/work-update	Bearer JWT (Engineer)	Submits geolocated photo proof and status update.
GET /api/admin/metrics	Bearer JWT (Admin)	Aggregated municipal performance indicators & SLA compliance.

Security and Integrity Protections

- 1. Firebase JWT Token Verification: Cryptographic verification of user UID and custom claims on every protected API call.
- 2. Parameterized SQL Queries: Supabase PostgREST and DAO layer guarantee zero SQL injection vulnerabilities.
- 3. Input Sanitization & Content Verification: Image uploads and text fields are sanitized and checked against size limits.
- 4. Immutable Audit Logs: Every status update and re-assignment generates a permanent audit trail entry in the database.

